

LEKCJA 27

Lekcja 27: Ciągła integracja. Część 3

Na tej lekcji studenci poznają:

<LearnIT/>

learnit.com.pl

Spis treści

- 1 Informacje o lekcji
- 2 Wymagania wstępne
- 3 Opis działania agentów Jenkins
- 4 Instalacja i konfiguracja agentów Jenkins
- 5 Uruchamianie kompilacji i zarządzanie agentami
- 6 Zaawansowane strategie wdrażania
- 7 Słownik pojęć lekcji
- 8 Podsumowanie lekcji
- 9 Część praktyczna
- 10 Zadania domowe
- 11 Kryteria oceny
- 12 Najlepsze praktyki i wskazówki
- 13 Rozwiązywanie problemów
- 14 Dodatki i rozszerzenia

1. Informacje o lekcji

Czas trwania: 180 minut **Poziom trudności:** Średniozaawansowany **Grupa docelowa:** Programiści, DevOps, administratorzy systemów, inżynierowie CI/CD

Cele lekcji

1. Opis działania agentów Jenkins: główne funkcje, wdrożenie na różnych platformach, konfigurowanie zabezpieczeń, elastyczność
2. Instalacja i konfiguracja agentów Jenkins w różnych środowiskach
3. Uruchamianie kompilacji na agentach i zarządzanie nimi
4. Najlepsze praktyki bezpieczeństwa i skalowania

2. Wymagania wstępne

- Lekcje 25-26: Podstawy CI/CD, Jenkins, Jenkins Pipeline i Groovy
 - Linux/Ubuntu (dla agentów SSH)
 - Docker (dla agentów kontenerowych)
 - Zainstalowany i skonfigurowany Jenkins master
 - Zrozumienie sieciowej komunikacji SSH i JNLP
-

Chcesz zostać DevOps Engineer?

Ten materiał to fragment kursu DevOps Engineer Online. Dołącz do nas i zdobądź praktyczne umiejętności pod okiem ekspertów z branży.

✓ Zajęcia na żywo 2x/tydzień ✓ Projekty praktyczne ✓ Certyfikat ukończenia

[Zobacz szczegóły kursu →](#)

3. Opis działania agentów Jenkins

1.1 Wprowadzenie do architektury agentów

Agenci Jenkins są kluczowym elementem rozproszonej architektury Jenkins, która pozwala na skalowanie i izolację środowisk kompilacji. Architektura Jenkins jest zaprojektowana dla rozproszonych środowisk kompilacji. Pozwala na używanie różnych środowisk dla każdego projektu kompilacji, równoważąc obciążenie między wieloma agentami uruchamiającymi zadania równolegle.

Główne komponenty architektury:

- **Kontroler Jenkins (Master)** - zarządza konfiguracją, orkiestruje pracę agentów i planuje zadania
- **Agenci Jenkins (Nodes)** - wykonują rzeczywiste zadania kompilacji w izolowanych środowiskach
- **Wykonawcy (Executors)** - procesy w ramach agentów odpowiedzialne za uruchamianie konkretnych zadań

1.2 Główne cechy agentów Jenkins

1. Rozłożenie obciążenia i skalowalność

Używanie agentów do wykonywania kompilacji zamiast uruchamiania ich na kontrolerze oferuje zwiększone bezpieczeństwo i skalowalność. Agenci Jenkins pozwalają na:

- **Efektywne wykorzystanie zasobów obliczeniowych** - rozpowszechnienie zadań na wiele maszyn
- **Skrócenie czasu realizacji zadań** - równoległe wykonywanie wielu kompilacji
- **Automatyczne skalowanie** - dynamiczne dodawanie i usuwanie agentów w zależności od obciążenia

```
// Przykład konfiguracji automatycznego skalowania
pipeline {
    agent {
        kubernetes {
            yaml """
                apiVersion: v1
                kind: Pod
                spec:
                    containers:
                    - name: maven
                      image: maven:3.8.6-openjdk-11
                      command:
                      - sleep
                    args:
                    - infinity
            """
        }
    }
    stages {
        stage('Build') {
            steps {
                container('maven') {
                    sh 'mvn clean compile'
                }
            }
        }
    }
}
```

2. Autonomia i niezawodność

Agenci Jenkins mogą działać autonomicznie, kontynuując wykonywanie zadań nawet w przypadku czasowej niedostępności kontrolera. Kluczowe aspekty:

- **Odporność na awarie** - kontynuacja pracy przy przerwach komunikacji
- **Cache lokalny** - przechowywanie artefaktów i zależności
- **Reconnection automatyczny** - automatyczne ponowne połączenie z kontrolerem

3. Elastyczność i customizacja

Agentów można skonfigurować do wykonywania różnych typów zadań:

- **Różne systemy operacyjne** - Windows, Linux, macOS
- **Specjalizowane narzędzia** - Docker, Kubernetes, cloud providers
- **Niestandardowe środowiska** - specyficzne wersje języków programowania, biblioteki

4. Bezpieczeństwo i izolacja

Aby włączyć najlepsze praktyki Jenkins, należy chronić węzeł kontrolera poprzez odmowę lub ograniczenie wykonywania kodu. Domyślnie Jenkins daje węzłowi kontrolera szerokie uprawnienia i pozwolenia—co czyni go celem o wysokiej wartości.

Kluczowe aspekty bezpieczeństwa:

- **Izolacja środowisk** - każdy agent działa w oddzielnym środowisku
- **Ograniczone uprawnienia** - agenci mają dostęp tylko do niezbędnych zasobów
- **Szyfrowana komunikacja** - bezpieczne połączenia między kontrolerem a agentami
- **Uwierzytelnianie** - konfiguracja kluczy SSH lub certyfikatów

1.3 Typy agentów Jenkins

Agenci stałe (Permanent Agents)

- Długotrwałe połączenia z kontrolerem
- Idealne dla środowisk o stałych wymaganiach
- Wymagają ręcznej konfiguracji i zarządzania

Agenci dynamiczne (Dynamic Agents)

- Tworzone na żądanie i usuwane po zakończeniu zadania
- Oszczędność zasobów i kosztów
- Lepsze dla środowisk cloud i kontenerowych

Agenci kontenerowe (Container Agents)

- Wykorzystują Docker lub Kubernetes
- Szybkie uruchamianie i niszczenie
- Izolacja na poziomie kontenera

1.4 Środowiska hybrydowe i multi-cloud

Agentów Jenkins można skonfigurować do pracy ze środowiskami hybrydowymi, takimi jak chmury, kontenery lub serwery fizyczne, umożliwiając tworzenie i wdrażanie aplikacji w wielu środowiskach.

Przykłady implementacji:

- **On-premise + Cloud** - część agentów w lokalnym data center, część w chmurze
- **Multi-cloud** - agenci rozłożeni między różnych dostawców chmury
- **Hybrid Kubernetes** - kombinacja klastrów lokalnych i cloud

4. Instalacja i konfiguracja agentów Jenkins

2.1 Przygotowanie środowiska

Wymagania systemowe dla agentów:

Minimalne wymagania:

- **RAM:** 1 GB (4 GB zalecane)
- **CPU:** 1 rdzeń (2 rdzenie zalecane)
- **Dysk:** 2 GB wolnego miejsca (10 GB zalecane)
- **Java:** OpenJDK 11 lub 17 (LTS)

Konfiguracja sieciowa:

- Dostęp do kontrolera Jenkins (zazwyczaj port 8080)
- Port SSH (22) lub JNLP (50000)
- Porty dla Docker (32768-60999 dla dynamicznego mapowania)

2.2 Instalacja agenta SSH na Ubuntu 22.04 LTS

```
# 1. Aktualizacja systemu
sudo apt update && sudo apt upgrade -y

# 2. Instalacja Java (OpenJDK 17 LTS)
sudo apt install openjdk-17-jdk -y
java -version

# 3. Tworzenie użytkownika jenkins
```

```

sudo useradd -m -s /bin/bash jenkins
sudo usermod -aG sudo jenkins

# 4. Konfiguracja SSH
sudo mkdir -p /home/jenkins/.ssh
sudo chown jenkins:jenkins /home/jenkins/.ssh
sudo chmod 700 /home/jenkins/.ssh

# 5. Generowanie kluczy SSH na kontrolerze
ssh-keygen -t rsa -b 4096 -f ~/.ssh/jenkins_agent_key -N ""

# 6. Kopiowanie klucza publicznego na agenta
sudo tee /home/jenkins/.ssh/authorized_keys << EOF
# Tutaj wklej zawartość pliku jenkins_agent_key.pub
EOF

sudo chown jenkins:jenkins /home/jenkins/.ssh/authorized_keys
sudo chmod 600 /home/jenkins/.ssh/authorized_keys

# 7. Konfiguracja zapory sieciowej
sudo ufw allow OpenSSH
sudo ufw allow from [IP_KONTROLERA_JENKINS]
sudo ufw enable

```

2.3 Konfiguracja agentów Docker

Metoda 1: Docker SSH Agent

```

# Generowanie kluczy SSH
ssh-keygen -f ~/.ssh/jenkins_agent_key -t rsa -b 4096 -N ""

# Uruchomienie kontenera agenta
docker run -d --rm --name=jenkins-agent-1 \
  -p 22:22 \
  -e "JENKINS_AGENT_SSH_PUBKEY=$(cat ~/.ssh/jenkins_agent_key.pub)" \
  jenkins/ssh-agent:alpine-jdk17

# Sprawdzenie stanu kontenera
docker ps
docker logs jenkins-agent-1

```


Metoda 2: Docker Compose

```
# docker-compose.yml
version: '3.8'

services:
  jenkins-controller:
    image: jenkins/jenkins:2.440-jdk17
    container_name: jenkins-controller
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    networks:
      - jenkins

  jenkins-agent:
    image: jenkins/ssh-agent:alpine-jdk17
    container_name: jenkins-agent
    environment:
      - JENKINS_AGENT_SSH_PUBKEY=${JENKINS_AGENT_SSH_PUBKEY}
    networks:
      - jenkins

networks:
  jenkins:
    driver: bridge

volumes:
  jenkins_home:
```

2.4 Konfiguracja agentów Kubernetes

Instalacja pluginu Kubernetes

```
# Instalacja przez Jenkins CLI
java -jar jenkins-cli.jar -s http://localhost:8080/ install-plugin kubernetes -restart
```

Konfiguracja YAML dla agenta Kubernetes

```
# jenkins-agent-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    jenkins/agent-type: "kubernetes"
spec:
  serviceAccountName: jenkins-agent
  containers:
    - name: jnlp
      image: jenkins/inbound-agent:latest-jdk17
      args: ['^${computer.jnlpMac}', '^${computer.name}']
      resources:
        requests:
          memory: "256Mi"
          cpu: "100m"
        limits:
          memory: "512Mi"
          cpu: "200m"
    - name: maven
      image: maven:3.8.6-openjdk-17
      command:
        - sleep
      args:
        - infinity
      resources:
        requests:
          memory: "512Mi"
          cpu: "200m"
        limits:
          memory: "1Gi"
          cpu: "500m"
    - name: docker
      image: docker:24-dind
      securityContext:
        privileged: true
      volumeMounts:
        - name: docker-sock
          mountPath: /var/run/docker.sock
  volumes:
    - name: docker-sock
```

```
hostPath:  
  path: /var/run/docker.sock
```

2.5 Konfiguracja w interfejsie Jenkins

Dodawanie stałego agenta SSH:

1. Zarządzanie węzłami:

- Przejdź do "Manage Jenkins" > "Manage Nodes and Clouds"
- Kliknij "New Node"

1. Konfiguracja podstawowa:

```
Node name: agent-ubuntu-01  
Type: Permanent Agent  
Number of executors: 2  
Remote root directory: /home/jenkins  
Labels: linux ubuntu java17  
Usage: Use this node as much as possible  
Launch method: Launch agents via SSH
```

1. Konfiguracja SSH:

```
Host: [IP_ADRES_AGENTA]  
Credentials: [jenkins-ssh-key]  
Host Key Verification Strategy: Non verifying Verification Strategy
```

1. Zaawansowane opciones:

```
JavaPath: /usr/bin/java  
JVM Options: -Xmx512m  
Prefix Start Agent Command:  
Suffix Start Agent Command:  
Connection Timeout in Seconds: 210  
Maximum Number of Retries: 10
```

Chcesz zostać DevOps Engineer?

Ten materiał to fragment kursu DevOps Engineer Online. Dołącz do nas i zdobądź praktyczne umiejętności pod okiem ekspertów z branży.

✓ Zajęcia na żywo 2x/tydzień ✓ Projekty praktyczne ✓ Certyfikat ukończenia

Zobacz szczegóły kursu →

5. Uruchamianie kompilacji i zarządzanie agentami

3.1 Uruchamianie zadań na określonych agentach

W Freestyle Project:

```
Restrict where this project can be run: [ENABLED]  
Label Expression: linux && java17
```

W Pipeline (Declarative):

```
pipeline {  
  agent {  
    label 'linux && java17'  
  }  
  stages {  
    stage('Build') {  
      steps {  
        sh 'echo "Building on agent: $NODE_NAME"'  
        sh 'java -version'  
        sh 'mvn clean package'      }  
    }  
  }  
}
```

```

    }
  }
}

```

W Pipeline z wieloma agentami:

```

pipeline {
  agent none

  stages {
    stage('Build') {
      agent { label 'java17' }
      steps {
        sh 'mvn clean compile'
      }
    }

    stage('Test') {
      parallel {
        stage('Unit Tests') {
          agent { label 'linux' }
          steps {
            sh 'mvn test'
          }
        }
        stage('Integration Tests') {
          agent { label 'docker' }
          steps {
            script {
              docker.image('postgres:14').withRun() { db ->
                sh 'mvn integration-test'
              }
            }
          }
        }
      }
    }

    stage('Deploy') {
      agent { label 'deployment' }
      when {
        branch 'main'
      }
    }
  }
}

```

```

        steps {
            sh 'kubectl apply -f k8s/'
        }
    }
}

```

3.2 Zarządzanie agentami Kubernetes

Pipeline z dynamicznym agentem:

```

pipeline {
    agent {
        kubernetes {
            yaml """
                apiVersion: v1
                kind: Pod
                spec:
                    serviceAccountName: jenkins-agent
                    containers:
                        - name: maven
                          image: maven:3.8.6-openjdk-17
                          command:
                              - sleep
                          args:
                              - infinity
                          resources:
                              requests:
                                  memory: 1Gi
                                  cpu: 500m
                              limits:
                                  memory: 2Gi
                                  cpu: 1000m
                        - name: docker
                          image: docker:24-cli
                          command:
                              - sleep
                          args:
                              - infinity
                          volumeMounts:
                              - mountPath: /var/run/docker.sock
                                name: docker-sock
                    volumes:

```

```

        - name: docker-sock
          hostPath:
            path: /var/run/docker.sock
      ""
    }
  }

  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }

    stage('Build') {
      steps {
        container('maven') {
          sh 'mvn clean package -DskipTests'
        }
      }
    }

    stage('Test') {
      steps {
        container('maven') {
          sh 'mvn test'
        }
      }
    }

    stage('Docker Build') {
      steps {
        container('docker') {
          sh 'docker build -t myapp:${BUILD_NUMBER} .'
          sh 'docker push myapp:${BUILD_NUMBER}'
        }
      }
    }
  }

  post {
    always {
      echo 'Pipeline completed on Kubernetes agent'
    }
  }

```

```
}  
}
```

3.3 Monitorowanie i raportowanie agentów

Skrypt Groovy do monitorowania agentów:

```
// Wykonaj w Jenkins Script Console  
import jenkins.model.*  
import hudson.model.*  
import hudson.slaves.*  
  
def jenkins = Jenkins.getInstance()  
  
println "=== AGENT STATUS REPORT ==="  
println "Total agents: ${jenkins.getNodes().size()}"  
println ""  
  
jenkins.getNodes().each { node ->  
    def computer = node.getComputer()  
    def status = computer.isOnline() ? "ONLINE" : "OFFLINE"  
    def executors = node.getNumExecutors()  
    def busyExecutors = computer.countBusy()  
    def freeExecutors = executors - busyExecutors  
  
    println "Agent: ${node.getDisplayName()}"  
    println "  Status: ${status}"  
    println "  Executors: ${executors} (Busy: ${busyExecutors}, Free: ${freeExecutors})"  
    println "  Labels: ${node.getLabelString()}"  
  
    if (!computer.isOnline()) {  
        def offlineCause = computer.getOfflineCause()  
        println "  Offline reason: ${offlineCause}"  
    }  
  
    println ""  
}
```


3.4 Najlepsze praktyki zarządzania agentami

1. Bezpieczeństwo agentów

Ustawienie uprawnień agentów w ten sposób pozwala ograniczyć dostęp agentów do serwera Jenkins i zasobów. Na przykład, można ograniczyć agentów do pisania tylko w określonych katalogach lub uruchamiania tylko określonych poleceń.

```
# Ograniczenie uprawnień użytkownika jenkins na agencie
sudo usermod -G jenkins jenkins # Usuń z grupy sudo
sudo chown -R jenkins:jenkins /home/jenkins
sudo chmod 750 /home/jenkins
```

2. Automatyzacja konfiguracji

```
// Configuration as Code dla agentów
jenkins:
  nodes:
    - permanent:
        name: "linux-agent-01"
        numExecutors: 2
        labelString: "linux java17 maven"
        launcher:
          ssh:
            host: "192.168.1.100"
            credentialsId: "jenkins-ssh-key"
            sshHostKeyVerificationStrategy: "nonVerifyingKeyVerificationStrategy"
        nodeProperties:
          - envVars:
              env:
                - key: "JAVA_HOME"
                  value: "/usr/lib/jvm/java-17-openjdk-amd64"
```

3. Monitoring i alerting

```
// Skrypt do sprawdzania zdrowia agentów
pipeline {
  agent { label 'master' }

  triggers {
```

```

    cron('H/15 * * * *') // Sprawdzaj co 15 minut
  }

  stages {
    stage('Check Agent Health') {
      steps {
        script {
          def offlineAgents = []

          Jenkins.instance.nodes.each { node ->
            def computer = node.computer
            if (!computer.isOnline()) {
              offlineAgents << node.displayName
            }
          }

          if (offlineAgents) {
            error "Offline agents detected: ${offlineAgents.join(', ')}"
          }
        }
      }
    }
  }

  post {
    failure {
      emailx (
        subject: "Jenkins Agent Health Check Failed",
        body: "Some Jenkins agents are offline. Please check the Jenkins console.",
        to: "devops@company.com"
      )
    }
  }
}

```

6. Zaawansowane strategie wdrażania

4.1 GitOps - Zarządzanie infrastrukturą poprzez kod

GitOps to metodologia, w której Git jest jednym źródłem prawdy (single source of truth) dla zarówno kodu aplikacji, jak i infrastruktury (Infrastructure as Code). System CI/CD automatycznie synchronizuje stan

rzeczywisty z żądanym stanem zdefiniowanym w Git.

Architektura GitOps pipeline'u:

1. Developer pushuje zmiany do Git
2. CI pipeline testuje kod i buduje obraz Docker
3. CD system automatycznie aktualizuje manifesty deployment'u
4. GitOps operator (np. ArgoCD, Flux) synchronizuje klaster z Git
5. Zmiana jest automatycznie wdrażana na produkcji

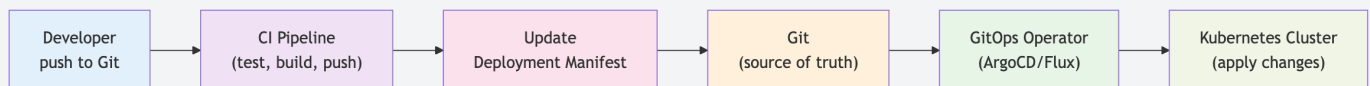


Diagram 1

Zalety GitOps:

- **Audit trail:** Wszystkie zmiany są śledzane w Git
- **Rollback:** Łatwy powrót do poprzedniej wersji poprzez Git revert
- **Transparency:** Widoczny dla całego zespołu stan infrastruktury
- **Compliance:** Zmiana żądana tylko przez Git, nie bezpośrednio na produkcji

4.2 Blue-Green Deployment - Zabezpieczone wdrażanie

Blue-Green to strategia wdrażania, gdzie dwa identyczne środowiska produkcyjne (Blue i Green) działają równolegle. Aktywne jest jedno (np. Blue), zaś drugie (Green) przechowuje nową wersję. Gdy Green jest gotowe i przetestowane, traffic jest przełączany do niego, a Blue staje się rezerwą.

Procedura Blue-Green Deployment:

1. Nowy kod jest wdrażany do Green environment
2. Przeprowadzane są testy i smoke tests
3. Traffic jest przełączany z Blue do Green
4. Blue environment pozostaje online na wypadek szybkiego rollback'u

Jeśli coś pójdzie nie tak, traffic jest przełączany z powrotem do Blue natychmiast.

4.3 Canary Deployment - Stopniowe wdrażanie

Canary deployment to strategia, gdzie nowa wersja jest wdrażana dla małej części użytkowników (5-10%), a jeśli nie ma problemów, stopniowo skalowana do 100%.

Procedura Canary Deployment:

1. 5% traffic przechodzi do nowej wersji
2. Monitorowanie metryk i błędów
3. Jeśli wszystko OK, zwiększ do 25%
4. Powtarzaj aż do 100%

Korzyści:

- Wczesne odkrycie problemów
- Minimalne ryzyko dla użytkowników
- Możliwość szybkiego rollback'u

KURS DEVOPS ENGINEER ONLINE

Chcesz zostać DevOps Engineer?

Ten materiał to fragment kursu DevOps Engineer Online. Dołącz do nas i zdobądź praktyczne umiejętności pod okiem ekspertów z branży.

✓ Zajęcia na żywo 2x/tydzień ✓ Projekty praktyczne ✓ Certyfikat ukończenia

Zobacz szczegóły kursu →

7. Słownik pojęć lekcji

Jenkins Agent (Agent Jenkinsa)

Maszyna fizyczna lub wirtualna połączona z Jenkinsem master'em, na której wykonywane są zadania pipeline'ów, umożliwiającą rozproszone i skalowalne przetwarzanie.

Node (Węzeł)

W Jenkinsie termin używany zamiennie z "agent" – pojedynczy komputer lub instancja VM, która jest częścią klastra Jenkinsa i wykonuje prace.

Distributed builds (Budowanie rozproszone)

Architektura Jenkinsa, w której master orkiestruje, a agenci wykonują zadania na różnych maszynach, umożliwiającą paralelizację i skalowanie.

Blue Ocean (Interfejs Blue Ocean)

Nowoczesny interfejs graficzny dla Jenkinsa, upraszczający wizualizację pipeline'ów i oferujący lepszą user experience niż klasyczny interfejs.

Pipeline as Code (Pipeline jako kod)

Praktyka definiowania całego procesu CI/CD w plikach Jenkinsfile (Groovy), przechowywanych w repozytorium, zamiast konfiguracji przez UI.

Shared Library (Biblioteka wspólna)

Reużywalny kod Groovy w Jenkinsie, definiujący wspólne funkcje, klasy i procedury, których można używać w wielu pipeline'ach.

Security (Bezpieczeństwo)

Praktyki i mechanizmy ochrony Jenkinsa: izolacja agentów, SSH keys zamiast haseł, ograniczone uprawnienia, szyfrowanie danych wrażliwych.

RBAC (Role-Based Access Control)

System kontroli dostępu w Jenkinsie, przydzielający użytkownikom role (admin, developer, viewer) z odpowiednimi uprawnieniami do zarządzania pipeline'ami i job'ami.

Jenkins Controller (Kontroler Jenkinsa)

Główny serwer/master w klastrze Jenkinsa, który orkiestruje prace, zarządza konfiguracją, prowadzi audit logs i komunikuje się z agentami.

8. Podsumowanie lekcji

- **Architektura rozproszonych agentów:** Controller (master) orkiestruje, agenci wykonują zadania równolegle w izolowanych środowiskach
- **Typy agentów:** stałe (permanent) dla długoterminowych, dynamiczne dla chmury, kontenerowe dla szybkiego skalowania
- **Agenci SSH:** Komunikują się z kontrolerem przez SSH z uwierzytelnianiem klawiszami
- **Agenci Docker:** Kontenery są uruchamiane na żądanie, oszczędzając zasoby
- **Agenci Kubernetes:** Idealne dla środowisk cloud-native z automatycznym skalowaniem
- **Bezpieczeństwo:** Izolacja, ograniczone uprawnienia, SSH keys zamiast haseł, brak kodu na masterze
- **Best practices:** Dedykowana infrastruktura dla agentów, monitoring zasobów, load balancing

- **GitOps:** Zarządzaj infrastrukturą poprzez Git jako jedno źródło prawdy
 - **Deployment Strategies:** Blue-Green, Canary i Rolling deployment dla bezpiecznego wdrażania
-

9. Część praktyczna

Zadanie 1: Konfiguracja agenta

W tym ćwiczeniu skonfigurujesz agenta Jenkins komunikującego się przez SSH.

Kroki do wykonania:

1. Uruchom dodatkową maszynę wirtualną Ubuntu 22.04.

1. Zainstaluj wymagane oprogramowanie:

```
sudo apt update
sudo apt install -y openssh-server openjdk-21-jre sudo
sudo service ssh start

# Utwórz użytkownika jenkins
sudo useradd -m -s /bin/bash jenkins
sudo echo 'jenkins:jenkins123' | sudo chpasswd
sudo usermod -aG sudo jenkins
```

1. Dodaj agenta w Jenkins:

- Zarządzaj Jenkinsem (zębatka) > Nodes > New Node
- Nazwa: "practice-agent"
- Typ: "Stały Agent"
- Kliknij "Create"

1. Skonfiguruj agenta:

- Zdalny system plików: "/home/jenkins/jobs"
- Metoda uruchomienia: "Launch agent by connecting it to the controller"
- Po wejściu na węzeł pojawią się komendy do wykonania na maszynie agenta

1. Utwórz testowy projekt:

```
pipeline {
    agent { label 'practice-agent' }
    stages {
        stage('Test') {
            steps {
                sh 'hostname'
                sh 'whoami'
                sh 'java -version'
                sh 'pwd'
            }
        }
    }
}
```

Oczekiwany rezultat: Działający agent wykonujący zadania w izolowanym środowisku.

Zadanie 2: Konfiguracja agentów Docker

W tym ćwiczeniu skonfigurujesz dynamicznych agentów wykorzystujących kontenery Docker.

Kroki do wykonania:

1. Zainstaluj Docker Plugin w Jenkins:

- Zarządzaj Jenkinsem > Plugins
- Wyszukaj i zainstaluj "Docker"

1. Skonfiguruj Docker Host:

```
# Na hoście z Dockerem
sudo mkdir -p /etc/systemd/system/docker.service.d
sudo tee /etc/systemd/system/docker.service.d/override.conf <<EOF
[Service]
ExecStart=
ExecStart=/usr/bin/dockerd -H fd:// -H tcp://0.0.0.0:4243
EOF

sudo systemctl daemon-reload
sudo systemctl restart docker
```


1. Dodaj Cloud w Jenkins:

- Zarządzanie Jenkinsem > Chmura > New cloud
- Add a new cloud > Docker
- Docker Host URI: `tcp://[IP_HOSTA]:4243`
- Test Connection

1. Skonfiguruj Docker Agent Template:

```
Labels: docker-agent
Enabled: checked
Name: docker-agent-template
Docker Image: maven:3.9.6-eclipse-temurin-21
Instance Capacity: 2
```

1. Utwórz Pipeline wykorzystujący Docker Agent:

```
pipeline {
    agent {
        label 'docker-agent'
    }
    stages {
        stage('Build') {
            steps {
                sh 'mvn --version'
            }
        }
    }
}
```

Oczekiwany rezultat: Dynamicznie tworzone i niszczone kontenery Docker służące jako agenci Jenkins.

Zadanie 3: Monitoring i zarządzanie agentami

W tym ćwiczeniu implementujesz monitoring zdrowia agentów i automatyczne zarządzanie nimi.

Kroki do wykonania:

1. Dodaj plugin "Pipeline Utility Steps"

2. Utwórz Pipeline monitorujący (odznacz Use Groovy Sandbox i zatwierdź skrypt):

```
pipeline {
    agent { label 'built-in' }

    triggers {
        cron('H/5 * * * *') // Co 5 minut
    }

    stages {
        stage('Agent Health Check') {
            steps {
                script {
                    def agentReport = [:]
                    def offlineAgents = []

                    Jenkins.instance.nodes.each { node ->
                        def computer = node.computer
                        def status = computer.isOnline() ? "ONLINE" : "OFFLINE"
                        def executors = node.numExecutors
                        def busyExecutors = computer.countBusy()

                        agentReport[node.displayName] = [
                            status: status,
                            executors: executors,
                            busy: busyExecutors,
                            free: executors - busyExecutors
                        ]

                        if (!computer.isOnline()) {
                            offlineAgents << node.displayName
                        }
                    }

                    // Zapisz raport jako artefakt
                    writeJSON file: 'agent-report.json', json: agentReport
                    archiveArtifacts artifacts: 'agent-report.json'

                    // Alert przy offline agentach
                    if (offlineAgents) {
                        currentBuild.result = 'UNSTABLE'
                        echo "WARNING: Offline agents detected: ${offlineAgents.join(',
                    ')}"}"
                }
            }
        }
    }
}
```



```

def createAgentMetrics() {
    def metrics = [:]
    def jenkins = Jenkins.get()

    // Pobieramy wszystkie komputery (węzły), w tym kontroler
    def allComputers = jenkins.getComputers()

    metrics.totalAgents = allComputers.size()
    metrics.onlineAgents = 0
    metrics.offlineAgents = 0
    metrics.totalExecutors = 0
    metrics.busyExecutors = 0
    metrics.agents = []

    allComputers.each { computer ->
        def isOnline = computer.isOnline()
        def name = computer.getDisplayName()

        if (isOnline) {
            metrics.onlineAgents++
        } else {
            metrics.offlineAgents++
        }

        def executors = computer.getNumExecutors()
        def busy = isOnline ? computer.countBusy() : 0

        metrics.totalExecutors += executors
        metrics.busyExecutors += busy

        // Zbieranie szczegółów o każdym agencie
        metrics.agents << [
            name: name,
            status: isOnline ? "ONLINE" : "OFFLINE",
            executors: executors,
            busy: busy,
            free: executors - busy,
            labels: computer.node ? computer.node.getLabelString() : "built-in"
        ]
    }

    // BEZPIECZNE ZAOKRĄGLANIE (Rozwiązanie Twojego błędu)
    if (metrics.totalExecutors > 0) {
        // Obliczamy procent jako BigDecimal
        BigDecimal utilization = new BigDecimal(metrics.busyExecutors)
    }
}

```

```

        .divide(new BigDecimal(metrics.totalExecutors), 4, RoundingMode.HALF_UP)
        .multiply(new BigDecimal(100))

        // Ustawiamy 2 miejsca po przecinku
        metrics.utilizationPercent = utilization.setScale(2, RoundingMode.HALF_UP)
    } else {
        metrics.utilizationPercent = 0
    }

    return metrics
}

// WYŚWIETLANIE WYNIKÓW
def m = createAgentMetrics()
println "===== "
println "                JENKINS AGENT DASHBOARD                "
println "===== "
println " Statystyki ogólne:"
println " - Wszystkie węzły:   ${m.totalAgents}"
println " - Online:              ${m.onlineAgents}"
println " - Offline:             ${m.offlineAgents}"
println " - Utylizacja:          ${m.busyExecutors}/${m.totalExecutors} executorów
(${m.utilizationPercent}%"
println "-----"
println " Szczegóły agentów:"
println String.format("%-20s | %-8s | %-10s | %s", "Nazwa", "Status", "Zajęte",
"Etykiety")

m.agents.each { a ->
    println String.format("%-20s | %-8s | %-10s | %s",
        a.name, a.status, "${a.busy}/${a.executors}", a.labels)
}
println "===== "

```

Oczekiwany rezultat: System monitorowania agentów i dashboardem pokazującym wykorzystanie zasobów.

10. Zadania domowe

Zadanie domowe 1: Implementacja i konfiguracja agenta na maszynie wirtualnej

Skonfiguruj dodatkowy węzeł (agent) Jenkinsa na osobnej maszynie wirtualnej (np. VirtualBox, VMware lub dowolny VPS) i zintegruj go z kontrolerem.

Wymagania:

1. Konfiguracja agenta (Static Agent): Przygotowanie VM: Skonfiguruj maszynę wirtualną z systemem Linux (np. Ubuntu). Zainstaluj na niej środowisko Java (JRE) zgodne z wersją na kontrolerze. **Połączenie:**

Skonfiguruj agenta. **2. Zarządzanie zasobami i Etykiety: Labels:** Nadaj agentowi unikalną etykietę (np.

`linux-worker-01`). **Environment:** Skonfiguruj na agencie zmienną środowiskową (np.

`NODE_ENV=production`) dostępną dla Pipeline'ów. **3. Pipeline testowy:** Utwórz Pipeline, który:

1. Wymusza uruchomienie na nowym agencie za pomocą parametru `agent { label 'linux-worker-01' }`.
2. Sprawdza parametry systemu (np. `sh 'uname -a'` , `sh 'free -m'`).
3. Weryfikuje obecność zmiennej środowiskowej zdefiniowanej w punkcie 2.

Zadanie domowe 2: Implementacja i konfiguracja automatycznie skalowanych agentów docker

Skonfiguruj docker'a jako źródło automatycznie skalowalnych agentów Jenkinsa.

Wymagania:

1. Konfiguracja docker: Przygotowanie Docker: Zainstaluj i skonfiguruj docker na maszynie wirtualnej tak, aby Jenkins mógł się do niej dobrać. **Połączenie:** Skonfiguruj Plugin Docker w Jenkins. **2. Pipeline testowy:**

Utwórz Pipeline, który:

1. Wymusza uruchomienie na agencie w dockerze.
2. Sprawdza parametry systemu (np. `cat /etc/os-release`).

11. Kryteria oceny

Poziom podstawowy (obowiązkowy):

- [] Zrozumienie architektury agentów Jenkins i ich roli w CI/CD
- [] Umiejętność instalacji i konfiguracji agentów
- [] Tworzenie i uruchamianie zadań na określonych agentach
- [] Podstawowe zarządzanie agentami przez interfejs Jenkins
- [] Wykonanie wszystkich zadań praktycznych

Poziom zaawansowany:

- [] Implementacja monitoringu i alertów dla agentów
- [] Zrozumienie zagadnień bezpieczeństwa agentów
- [] Automatyzacja zarządzania agentami (Configuration as Code)

12. Najlepsze praktyki i wskazówki

Bezpieczeństwo agentów

1. **Nigdy nie uruchamiaj kompilacji na kontrolerze** - używaj zawsze agentów
2. **Ograniczaj uprawnienia agentów** - zasada najmniejszych uprawnień
3. **Używaj szyfrowanej komunikacji** - SSH z kluczami lub TLS dla JNLP
4. **Regularne aktualizacje** - utrzymuj agentów w najnowszych wersjach
5. **Izolacja środowisk** - różni agenci dla różnych środowisk (dev/staging/prod)

Wydajność i skalowalność

1. **Monitoruj wykorzystanie zasobów** - CPU, pamięć, dysk
2. **Używaj labeli do kategoryzacji** - logiczne grupowanie agentów
3. **Implementuj cache'owanie** - dla zależności i artefaktów
4. **Automatyczne skalowanie** - dodawaj/usuwas agentów na żądanie
5. **Load balancing** - równomiernie rozdzielaj zadania

Zarządzanie konfiguracją

1. Configuration as Code - wszystko w kodzie, nic ręcznie
2. Wersjonowanie konfiguracji - tracking zmian w Git
3. Standardowe obrazy - jednolite środowiska dla podobnych zadań
4. Backup i recovery - plany odzyskiwania konfiguracji
5. Testing konfiguracji - walidacja przed wdrożeniem

13. Rozwiązywanie problemów

Typowe problemy z agentami

Agent nie może się połączyć:

```
# Sprawdź sieć
telnet <jenkins-controller> 50000
ping <jenkins-controller>

# Sprawdź porty
netstat -tlnp | grep :50000

# Sprawdź logi agenta
tail -f /var/log/jenkins/jenkins-agent.log
```

Problemy z SSH:

```
# Test połączenia SSH
ssh -i ~/.ssh/jenkins_agent_key -o StrictHostKeyChecking=no jenkins@<agent-ip>

# Sprawdź authorized_keys
cat /home/jenkins/.ssh/authorized_keys

# Sprawdź uprawnienia
ls -la /home/jenkins/.ssh/
```

Problemy z Kubernetes:


```
# Sprawdź pody agentów
kubectl get pods -l jenkins=agent

# Sprawdź logi
kubectl logs <jenkins-agent-pod>

# Sprawdź events
kubectl get events --sort-by=.metadata.creationTimestamp
```

14. Dodatki i rozszerzenia

Przydatne wtyczki Jenkins

1. **Kubernetes Plugin** - dynamiczne agenty na Kubernetes
2. **Docker Plugin** - agenci w kontenerach Docker
3. **Amazon EC2 Plugin** - agenci na AWS EC2
4. **Azure VM Plugin** - agenci na Azure
5. **Node and Label Parameter Plugin** - wybór agenta przez parametr
6. **Build Timeout Plugin** - limity czasowe dla zadań
7. **Monitoring Plugin** - metryki i monitoring agentów

Skrypty utility

Restart wszystkich offline agentów:

```
import jenkins.model.*
import hudson.slaves.*

Jenkins.instance.nodes.each { node ->
    def computer = node.computer
    if (computer.isOffline()) {
        println "Restarting ${node.displayName}..."
        computer.connect(false)
    }
}
```

Bulk update agentów:

```
import jenkins.model.*
import hudson.slaves.*

// Aktualizuj wszystkich agentów Linux
Jenkins.instance.nodes.findAll {
    it.labelString.contains("linux")
}.each { node ->
    node.numExecutors = 4 // Zwiększ liczbę executorów
    node.save()
    println "Updated ${node.displayName}"
}
```

15. Dodatkowe materiały

Polecana literatura

1. "Jenkins: The Definitive Guide" – John Ferguson Smart
2. "Kubernetes in Action" – Marko Lukša
3. "Docker Deep Dive" – Nigel Poulton
4. "Site Reliability Engineering" – Google SRE Team

Przydatne narzędzia

1. Jenkins Configuration as Code (JCascC) - <https://jenkins.io/projects/jcasc/>
2. Helm Charts for Jenkins - <https://charts.jenkins.io>
3. Jenkins Operator - <https://jenkinsci.github.io/kubernetes-operator/>
4. Prometheus Jenkins Plugin - monitoring i metryki
5. Grafana Dashboards - wizualizacja metryk Jenkins

Oficjalna dokumentacja

Poniższe zasoby są niezbędne do pełnego zrozumienia narzędzi CI/CD omówianych na tej lekcji:

1. GitHub Actions - <https://docs.github.com/en/actions>

- Dokumentacja zawiera szczegółowe przewodniki, best practices i przykłady workflow'ów

1. **GitLab CI/CD** - <https://docs.gitlab.com/ee/ci/>

- Kompleksowa dokumentacja dotycząca pipeline'ów, runners'ów i zaawansowanych funkcji

1. **Jenkins** - <https://www.jenkins.io/doc/>

- Oficjalna dokumentacja Jenkins z instrukcjami instalacji, konfiguracji i integracji

Linki i zasoby

1. **Oficjalna dokumentacja agentów Jenkins** - <https://www.jenkins.io/doc/book/using/using-agents/>

2. **Jenkins Kubernetes Plugin** - <https://plugins.jenkins.io/kubernetes>

3. **Best Practices for Jenkins** - <https://www.jenkins.io/doc/book/using/best-practices/>

4. **Jenkins Security** - <https://www.jenkins.io/doc/book/security/>

5. **Community Forum** - <https://community.jenkins.io/>

Checklista wdrożenia

Przed produkcją:

- [] Skonfigurowana komunikacja szyfrowana
- [] Ograniczone uprawnienia agentów
- [] Monitoring i alerty włączone
- [] Backup konfiguracji przygotowany
- [] Dokumentacja aktualna
- [] Testy load balancing przeprowadzone
- [] Plan disaster recovery przygotowany
- [] Security audit wykonany

Po wdrożeniu:

- [] Monitoring metryki zbierane
- [] Logi centralizowane
- [] Alerty działają poprawnie
- [] Performance baseline ustalony

- [] Użytkownicy przeszkoleni
- [] Dokumentacja operacyjna aktualna
- [] Procedury troubleshooting przetestowane

Powodzenia w implementacji agentów Jenkins! Pamiętaj, że dobrze skonfigurowana infrastruktura agentów to podstawa skalowalnego i niezawodnego systemu CI/CD.

ROZPOCZNIJ KARIERĘ W DEVOPS ENGINEER

Gotowy na następny krok?

Ten materiał to tylko początek. Dołącz do kursu DevOps Engineer Online i zdobądź praktyczne umiejętności pod okiem ekspertów.

- ✓ 5,5 miesiąca intensywnej nauki
- ✓ Zajęcia na żywo 2x w tygodniu
- ✓ Certyfikat + pomoc w znalezieniu pracy

Zobacz szczegóły kursu →

☎ Zadzwoń do nas

✉ info@learnit.com.pl

☎ +48 573-580-990

🌐 learnit.com.pl